

TRO — Deployable System Architecture & 1-Month Build Plan

This assumes a **real, running product** wired to Jira (and later Power BI, ServiceNow, etc.), not a paper prototype. Every choice below is picked for two things at once: (a) buildable by 3 people in ~4 weeks, and (b) an honest path to scaling to real MSP traffic without a rewrite.

1. Model / Architecture Choice Per Agent

Agent	What it needs to do	Model / Technique	Why this, not something fancier
Agent 1 — Intake & Extraction	Turn free text into intent + entities + completeness score	spaCy (en_core_web_trf) for NER/POS as a fast first pass, + one LLM call (Groq-hosted Llama 3.1 8B/70B, function-calling / JSON mode) to fill in what spaCy can't (intent, urgency, structured slot-filling)	spaCy is free, fast, runs on CPU, and handles boilerplate entity extraction without burning an LLM call on every token. The LLM only does the harder semantic part. No fine-tuning needed in month 1.
Agent 2 — Adaptive Clarifier	Decide the single best next question, track slot state	LLM call (same provider), stateful prompt carrying the running slot-fill JSON + a hard cap of 2–3 turns enforced in code, not by the LLM	This is genuinely agentic — it needs reasoning over partial state, not a lookup table. Capping turns in code (not prompt) protects the live demo from the "user won't answer" loop.
Agent 3 — KB Retrieval & Deflection Gate	Embed the query, retrieve top-k KB chunks, score resolution confidence, compose an answer	sentence-transformers all-MiniLM-L6-v2 for embeddings (free, CPU-friendly, 384-dim) → pgvector similarity search → LLM call to compose the offered fix in natural language	MiniLM is the standard cheap-and-good embedding choice; pgvector means you don't run a second database (see DB section) just for this.

Agent 4 — Correlation Engine	Duplicate/incident-cluster detection over the open-ticket pool	Same MiniLM embeddings + cosine similarity threshold (start at 0.85–0.88, sweep later) + a SQL time-window filter (last N hours)	This is deterministic threshold logic, not a model that needs training — your own doc already flags this honestly. Don't over-engineer it in month 1; ship the simplified version and note the full temporal-cluster version as a fast-follow.
Agent 5 — Classify, Prioritize, Explain	Category + priority classification, plain-English routing rationale	Classical ML baseline: TF-IDF + Logistic Regression (or LightGBM) trained on the Kaggle IT-ticket datasets for category/priority, not a fine-tuned transformer in month 1; LLM call generates the rationale text from the classifier's output + matched features	Classical ML trains in minutes on a laptop CPU, is reproducible, and is easy to explain in a viva/demo. A fine-tuned DistilBERT is a good stretch goal for week 4 if time allows, but don't block the pipeline on it.
Agent 6 — Ticket Creation & Dispatch	Populate canonical schema → call Jira REST API v3 → write audit record	Pure code, no model. httpx async client + Pydantic schema validation + retry/backoff	No ML here — this is deterministic integration code, as your own architecture doc already concludes.
Orchestrator	State machine across all 6 agents, gate logic	LangGraph (StateGraph) or a hand-rolled async state machine in FastAPI	LangGraph gives you the shared-state graph pattern for free and is well documented; a hand-rolled version is fine too if the team is more comfortable with plain Python and wants fewer dependencies to debug under time pressure. Pick LangGraph if at least one person has used it before — otherwise don't spend week 1 learning it.

LLM provider recommendation: Groq (Llama 3.1 8B/70B) as primary — fast, has a free tier, OpenAI-compatible API. Keep an OpenAI GPT-4o-mini fallback key for when you need

slightly better structured extraction. Don't run a local Ollama model as your primary path for a 1-month timeline — it adds hardware dependency and latency variance you don't have time to debug.

2. Backend Architecture

Framework: FastAPI, not Flask. Reasons specific to this project: native async (needed for concurrent LLM calls + Jira API calls without blocking), Pydantic models double as your canonical ticket schema validation, built-in WebSocket support for the live chat turn, and automatic OpenAPI docs which is genuinely useful when 3 people are integrating against each other's endpoints under time pressure.

Structural pattern: modular monolith, not microservices — for month 1. Your reference docs describe a full Docker microservices layout (one container per agent). That's the right *target* architecture, but building and debugging 8 separate containers with inter-service networking in a 4-week window is where teams lose a week to plumbing instead of building. Recommendation:

tro-backend/

```
├── app/
│   ├── agents/
│   │   ├── intake.py      # Agent 1
│   │   ├── clarifier.py   # Agent 2
│   │   ├── retrieval.py   # Agent 3
│   │   ├── correlation.py # Agent 4
│   │   ├── classifier.py  # Agent 5
│   │   └── dispatch.py    # Agent 6
│   ├── orchestrator/
│   │   └── graph.py        # LangGraph or state machine
│   ├── integrations/
│   │   ├── jira_client.py
│   │   └── llm_client.py  # provider-agnostic wrapper (Groq/OpenAI)
│   ├── models/            # Pydantic schemas (canonical ticket, etc.)
│   ├── db/
│   │   ├── postgres.py
│   │   └── redis_session.py
│   ├── api/
│   │   ├── chat.py        # WebSocket + REST chat endpoints
│   │   ├── dashboard.py   # analytics endpoints
│   │   └── auth.py
│   └── main.py
├── ml_training/           # offline training scripts, not part of the runtime API
│   ├── train_classifier.py
│   └── notebooks/
├── tests/
└── docker-compose.yml
```

└─ requirements.txt

Each agent is a **module with a clean function interface** (`async def run(state) -> state`), so splitting it into a real microservice later is a config change, not a rewrite — you get the migration path without paying for it now.

API surface:

- `WS /chat/{session_id}` — live conversation turns (Agents 1–3, synchronous with the user)
 - `POST /tickets` — internal, called by Agent 6 after gating
 - `GET /dashboard/*` — analytics reads
 - `POST /webhooks/jira` — passive webhook mode
 - Standard `/health`, `/metrics` for ops
-

3. Frontend Architecture

Two separate frontends with different urgency:

1. **Chat widget (user-facing) — React.** This is the actual product surface, so it should look and feel real: React + a WebSocket client, Tailwind for styling. Keep it a single embeddable component so it can later be dropped into Slack/Teams/a client's portal.
2. **Reviewer/analytics dashboard — Streamlit for month 1**, with a note that a React admin panel is the "real" version. Streamlit lets one person (per your existing role split, Person C) ship ticket-review, audit-trail viewing, and basic charts in days instead of a week+, which matters a lot in a 4-week timeline. Swap to a React admin dashboard once the MVP is validated.

Don't build the Power BI connector in month 1 — Streamlit reading directly from Postgres gets you the same demo value (deflection rate, confidence distributions, escalation rate) without provisioning Power BI, licensing, and a Postgres connector config under time pressure. Note this explicitly as a "swap-in later" item.

4. Training the ML Models — Technique

You are **not** training an LLM. You're training one small classical model (Agent 5) and using off-the-shelf embeddings everywhere else. Concretely:

1. **Data:** Kaggle ITSM/IT-ticket datasets (as listed in your reference docs) + a synthetic 100–300 ticket set matching your own canonical schema, so the model has seen your exact category taxonomy.

2. **Preprocessing:** lowercase, strip PII/emails before training (never train or log on real PII), TF-IDF vectorization (unigrams + bigrams, `max_features` ~5–10k).
3. **Model:** start with **Logistic Regression** or **LinearSVC** as the category classifier — trains in seconds, gives calibrated-enough probabilities for your confidence gate with `predict_proba`. Try **LightGBM** as a second candidate if you want a slightly stronger baseline; compare on held-out F1 and pick whichever wins, don't assume.
4. **Priority model:** can often be **rule-based** (impact + urgency keywords → priority tier) rather than a separate ML model — your own docs note classical rules may beat a learned model here for reproducibility. Use ML only if the rules clearly underperform on your eval set.
5. **Evaluation:** standard train/val/test split (70/15/15), report precision/recall/F1 per class, and do the **confidence calibration check** from your metrics table (bucket predicted confidence vs. actual accuracy) — cheap, and it directly validates whether your Confidence Gate threshold is trustworthy.
6. **Stretch goal (week 4 only, if ahead of schedule):** fine-tune a small transformer (DistilBERT) on the same data and compare F1 against the classical baseline. Only ship it if it's clearly better — otherwise the classical model is the more defensible choice for a 1-month build.
7. **Embeddings (Agents 3 & 4):** no training at all — `all-MiniLM-L6-v2` is used frozen, off the shelf.

This whole pipeline runs on CPU in minutes; you do not need a GPU for training in this scope.

5. Database

PostgreSQL as the single system of record — with the `pgvector` extension enabled.

Why one Postgres instance instead of Postgres + a separate ChromaDB/FAISS store: it removes an entire second system to deploy, secure, and back up, and for the traffic volumes here (conversation-scale, not web-scale), `pgvector`'s performance is more than sufficient. You get relational tables (tickets, audit log, users, KB metadata) and vector similarity search (KB embeddings, ticket embeddings for Agent 4) in one place, one connection pool, one backup strategy.

- **Tables:** `conversations`, `tickets` (canonical schema), `audit_trail`, `kb_articles` (with `vector` column), `users`, `escalations`.
- **Redis** — one addition, for **ephemeral conversation/session state only** (the in-flight slot-fill dict during a live chat turn, rate-limit counters). Don't put anything durable in Redis; Postgres is the source of truth.
- **MongoDB is not recommended here** — your data is fundamentally relational (tickets have fixed schema, foreign keys to users/audit logs) plus vector search, both of which Postgres+`pgvector` already covers well. Adding Mongo would mean a third system for no clear benefit.

6. Security Measures

Since this touches real user text, an ITSM system, and (eventually) real client data, treat security as non-optional even in the demo build:

- **Transport:** TLS everywhere (HTTPS/WSS) — no plaintext connections, including internal service calls once you split into containers.
- **Auth:** JWT-based auth for the dashboard/reviewer UI (OAuth2 password or client-credentials flow via FastAPI's built-in `OAuth2PasswordBearer`); a separate API key for service-to-service calls (e.g., Jira webhook verification using Jira's signing secret).
- **Secrets management:** `.env` + `python-dotenv` locally (never committed — enforce via `.gitignore` and a pre-commit hook), **AWS Secrets Manager / Azure Key Vault** in any deployed environment. No API keys in code or Docker images.
- **PII handling:** strip or mask direct identifiers (emails, employee IDs) before they're sent to a third-party LLM API where avoidable; if the LLM must see them for the ticket to make sense, disclose this explicitly in the paper's/product's data-handling notes. Never use real user data to train the classifier — synthetic/Kaggle only.
- **Data at rest:** enable Postgres encryption at rest (native on RDS/Azure Database for PostgreSQL, or disk-level encryption if self-hosted).
- **Access control:** row-level security in Postgres if/when you get to multi-tenant (MSP) mode, so Client A's tickets are structurally invisible to Client B's queries — flagged as a known gap to close before any real MSP pilot, per your own proposal.
- **Input validation:** every API boundary goes through Pydantic models — rejects malformed payloads before they reach an agent or the LLM.
- **Rate limiting:** basic per-session/per-IP rate limiting (e.g., `slowapi`) on the chat endpoint to prevent abuse and control LLM API cost.
- **Audit logging:** every gate decision, confidence score, and Jira dispatch is already part of your schema — make sure it's write-once/append-only in practice (no update/delete on `audit_trail` rows) so it's actually trustworthy as an audit trail.
- **Jira credentials:** stored as an API token (not a password) with the minimum scopes needed, rotated periodically.

7. Deployment — for Real Scalability

Two-phase recommendation, because "scalable" and "buildable in a month" pull in different directions if you try to do both at once on day one.

Phase 1 — Demo/MVP deployment (what you actually ship in week 4):

- `docker-compose` locally for dev/testing (Postgres, Redis, FastAPI backend, React frontend, Streamlit dashboard as services).

- Deploy the same containers to **Railway or Render** (free/cheap tier) for a publicly reachable demo URL — zero infra config, fast to get live, exactly matches your reference docs' "cheapest path" recommendation.
- Managed Postgres add-on from the same platform (avoids self-hosting a DB under time pressure).

Phase 2 — The scale-out path (documented, not necessarily built, in month 1):

- Containers → **AWS ECS Fargate** or **Azure Container Apps** (serverless containers, matches what your own docs already recommend) — no server management, built-in autoscaling per service.
- **Managed Postgres** (RDS / Azure Database for PostgreSQL) with read replicas once dashboard read traffic grows separately from write traffic.
- **API Gateway / Application Load Balancer** in front of the FastAPI service(s); split the WebSocket chat path from the REST dashboard path if load profiles diverge.
- Each agent module can be peeled out into its **own Fargate service** exactly along the module boundaries you already built in Phase 1 — this is why the modular-monolith structure above matters: the seams are already there.
- **Async task queue** (e.g., Celery + Redis, or a cloud-native queue like SQS) for anything that doesn't need to block the live chat turn — e.g., Agent 4's open-ticket-pool refresh, Power BI sync, non-urgent audit writes.
- **CI/CD**: GitHub Actions — lint/test on PR, build+push Docker images, deploy to staging automatically, prod on manual approval.
- **Observability**: start with basic structured logging + the audit trail in Postgres; add CloudWatch/Application Insights and per-stage latency dashboards (Grafana, per your own docs) once you're past the demo stage.
- **Multi-tenancy** (if pursued for real MSP clients): per-client vector namespaces in pgvector + Postgres row-level security, as your proposal already flags — don't attempt this in month 1, just don't build anything that structurally blocks it later (e.g., always include a `tenant_id` column from day one, even if you only ever populate it with one value).

8. Hardware Requirements

Because the LLM calls go through hosted APIs (Groq/OpenAI) rather than running a local model, hardware needs are modest — this is the single biggest simplification versus the "run Llama locally" path in your reference docs.

Development (per laptop):

- CPU: any modern quad-core+ (Intel i5/Ryzen 5 or better)
- RAM: 16GB comfortable, 8GB workable (Docker + Postgres + IDE gets tight below that)
- GPU: **not required** — you're not running local inference or training transformers from scratch; classical ML training (TF-IDF+LogReg/LightGBM) is CPU-only and finishes in minutes

- Disk: ~10GB free (Docker images, datasets, embeddings cache)

Production (Phase 2, cloud):

- Compute: small container instances per service — 1–2 vCPU / 2–4GB RAM is plenty for the FastAPI orchestrator and each lightweight agent module at demo/pilot traffic; scale horizontally (more instances) rather than vertically as load grows, which is the point of the Fargate/Container Apps choice.
- GPU: **not needed** in production either, since inference is via LLM APIs and the classifier is a small CPU model. Only revisit this if you later self-host a fine-tuned transformer at real scale, which is explicitly a post-graduation/stretch item in your own roadmap, not a month-1 concern.
- DB: managed Postgres instance sized to start small (e.g., `db.t3.medium`-class or equivalent) and scale as ticket/audit volume grows.

9. One-Month Timeline, 3 People

Keeping your existing role split (A = conversational/NLP, B = retrieval/classification/correlation, C = orchestrator/infra/integrations), because it already maps cleanly onto the agent boundaries.

Explicit scope cuts for month 1 (vs. the full 3–6 month plan in your proposal), stated up front so nothing is silently dropped:

- Agent 4 ships as **simplified threshold-only duplicate detection**, not full temporal clustering.
- Classifier is the **classical ML baseline**, not a fine-tuned transformer (transformer is a week-4 stretch goal only).
- Dashboard is **Streamlit**, not Power BI (Power BI is a documented "swap-in later," not built).
- No θ -sweep, no full evaluation suite, no confidence-calibration report — ship the pipeline working end-to-end and log everything needed to run these evaluations afterward.
- Deployment target is **Railway/Render demo**, not AWS ECS/Azure Container Apps (the scale-out path is documented, not built).

Week	Person A (Agents 1–2)	Person B (Agents 3–5)	Person C (Orchestrator, Agent 6, infra)
1 — Foundation	spaCy pipeline + LLM extraction prompt; conversation state schema	Collect Kaggle datasets + build synthetic ticket set; stand up Postgres+pgvector; build KB corpus	Postgres schema (tickets, audit, users); docker-compose skeleton; Jira sandbox +

			API token; repo/CI scaffolding
2 — Core agents	Agent 1 working end-to-end (text → structured fields); Agent 2 clarifier with turn cap	Agent 3 retrieval + deflection scoring; train Agent 5 classical classifier, evaluate F1	Orchestrator skeleton (LangGraph or state machine) wiring Agents 1–2; Agent 6 Jira dispatch (create issue via REST API v3)
3 — Integration	Integrate Agent 1–2 into orchestrator; add completeness gate logic	Agent 4 simplified correlation (embedding similarity + time window); wire Agent 5's rationale generation	Full pipeline wired end-to-end; Confidence Gate → human-review path; auth + secrets management; audit trail writes
4 — Polish & ship	Chat UI (React) hooked to WebSocket; bug fixes on clarification flow	Eval pass on classifier + deflection accuracy; tune thresholds	Streamlit dashboard (deflection rate, escalation rate, latency); deploy to Railway/Render; demo script + rehearsal

Cross-cutting, shared across all three throughout: integration testing every Friday (run 5–10 real conversations end-to-end through the whole pipeline, not just unit tests per agent), and starting IDF/IPR paperwork in week 1 in parallel if you're pursuing the patent angle — it has its own lead time independent of the code.

By end of week 4 you have: a working chat → clarify → deflect/correlate → classify → dispatch-to-Jira pipeline, a reviewer dashboard, basic security (auth + secrets + audit trail), and a deployed demo URL — which is a legitimate, honestly-scoped MVP, not a compressed version of the full 6-month vision pretending to be complete.

Tech Stack

Here's a concrete, cost-conscious stack, with free-tier limits noted so there are no surprises mid-build.

Reference UI : <https://helpdesk.vit.ac.in/>

Hosting on Vercel Free tier :

<https://www.programonaut.com/host-your-application-for-free-with-vercel-step-by-step/>

Frontend

React (Vite, not CRA) for the chat widget + **Tailwind CSS** for styling.

- **Why:** WebSocket support, component reuse for embedding into Slack/Teams later, huge free ecosystem.
- **Free:** entirely free/open-source. Hosting on Vercel or Netlify free tier.
- Dashboard: **Streamlit** (free, Python-native, fastest to ship) instead of a second React app for month 1.

Backend

FastAPI (Python) + **Uvicorn**.

- **Why:** async native (needed for concurrent LLM + Jira calls), Pydantic validation doubles as your schema, built-in WebSocket + OpenAPI docs, free and open-source.
- Task queue (if needed for async jobs): **Celery + Redis**, or **FastAPI BackgroundTasks** for lighter needs (no extra infra).
- <https://www.geeksforgeeks.org/python/fastapi-uvicorn/>

Database

PostgreSQL + pgvector extension as the single system of record (relational + vector search in one place).

- **Free hosting:** Supabase free tier (500MB DB, pgvector built-in, generous for a prototype), or Neon.tech free tier (serverless Postgres, also supports pgvector). Both are genuinely free at prototype scale.
- **Redis** for ephemeral session state only — Upstash free tier (Redis-as-a-service, pay-per-request, effectively free at low volume).
- Avoid a separate MongoDB/ChromaDB deployment — extra infra to secure and pay for with no real benefit over Postgres+pgvector at this scale.

LLM Models

Task	Model	Provider	Cost
Extraction, clarification, rationale generation	Llama 3.1 8B / 70B	Groq	Free tier (generous rate limits, very fast inference)
Fallback / higher-quality structured extraction	GPT-4o-mini	OpenAI	~\$0.15/1M input tokens — under \$1/month at prototype volume

Embeddings (KB retrieval + Agent 4 correlation)	<code>all-MiniLM-L6-v2</code>	sentence-transformers , runs locally	Free, CPU-only, no API cost at all
Classification (Agent 5)	TF-IDF + Logistic Regression / LightGBM	scikit-learn / LightGBM, self-trained	Free, trains on CPU in minutes

Wrap the LLM calls behind a provider-agnostic client (`llm_client.py`) so switching Groq ↔ OpenAI ↔ Gemini free tier is a config change, not a rewrite — useful since free tiers sometimes throttle under demo load.

Hosting

Component	Platform	Free tier
Backend (FastAPI) + Postgres + Redis	Railway or Render	Both have free/low-cost tiers (\$0–5/mo) sufficient for a demo; auto-deploys from GitHub
Frontend (React)	Vercel or Netlify	Free tier, generous for a prototype's traffic
Dashboard (Streamlit)	Streamlit Community Cloud	Free, one-click deploy from GitHub
Container registry (if using Docker)	GitHub Container Registry	Free for public/small private repos

This gets you a fully live, publicly-reachable demo at **\$0–5/month total**, matching the "cheapest possible" numbers your own reference docs already worked out.

Security

Authentication: JWT (`python-jose` or `PyJWT`) via FastAPI's `OAuth2PasswordBearer` for dashboard/reviewer login; short-lived access tokens + refresh tokens; API keys (stored hashed) for service-to-service calls like Jira webhooks.

- <https://medium.com/@amr2018/how-to-generate-jwt-token-using-python-36c2305c5a14>
- <https://pytutorial.com/python-jwt-implementation-guide-secure-authentication-made-easy/>

PII masking: run a regex/spaCy-based scrubber (emails, phone numbers, employee IDs) on user text **before** it's sent to any external LLM API — `presidio` (Microsoft's open-source PII

detection library, free) is a good fit here and saves you writing detection rules by hand. Store the unmasked version only in Postgres, behind auth, never in LLM prompts or logs.

Web security basics:

- HTTPS/WSS everywhere — free via Let's Encrypt (automatic on Railway/Render/Vercel).
- CORS locked to your known frontend origin(s) in FastAPI, not `*`.
- Rate limiting: `slowapi` (free, FastAPI-native) on the chat and dashboard endpoints.
- Input validation: Pydantic models on every endpoint — rejects malformed/oversized payloads before they hit an agent or the LLM.
- CSRF protection on any cookie-based session; prefer token-based auth (JWT in header) to sidestep this mostly.
- Dependency scanning: GitHub's free `Dependabot` alerts.

Encryption:

- **In transit: TLS** (free, handled by the hosting platforms above).
- **At rest:** Supabase/Neon/Railway managed **Postgres encrypts at rest by default** — **verify it's on**, don't build it yourself.
- **Secrets:** `.env` locally (gitignored) → **Doppler free tier** or the hosting platform's built-in secrets manager (Railway/Render both have one) in deployment — avoids hand-rolling secret injection.

Audit trail: append-only `audit_trail` table in Postgres (no UPDATE/DELETE grants on that table for the app's DB role) — cheap to enforce, makes the trail actually trustworthy.

Key Libraries Summary

```
fastapi, uvicorn[standard]
pydantic
python-jose[cryptography] # JWT
slowapi                   # rate limiting
sqlalchemy, asyncpg      # Postgres async ORM
pgvector                  # vector column support
redis                     # session state
spacy, sentence-transformers
scikit-learn, lightgbm    # classifier
presidio-analyzer         # PII masking
httpx                     # async HTTP (Jira, LLM APIs)
langgraph                 # orchestration (optional)
groq, openai              # LLM SDKs
python-dotenv
pytest, pytest-asyncio    # testing
```

Every tool above is either fully open-source/free or has a free tier that comfortably covers a prototype's traffic — the only place real money enters is LLM API overage past the free tiers, which stays under a few dollars a month at demo/pilot scale.